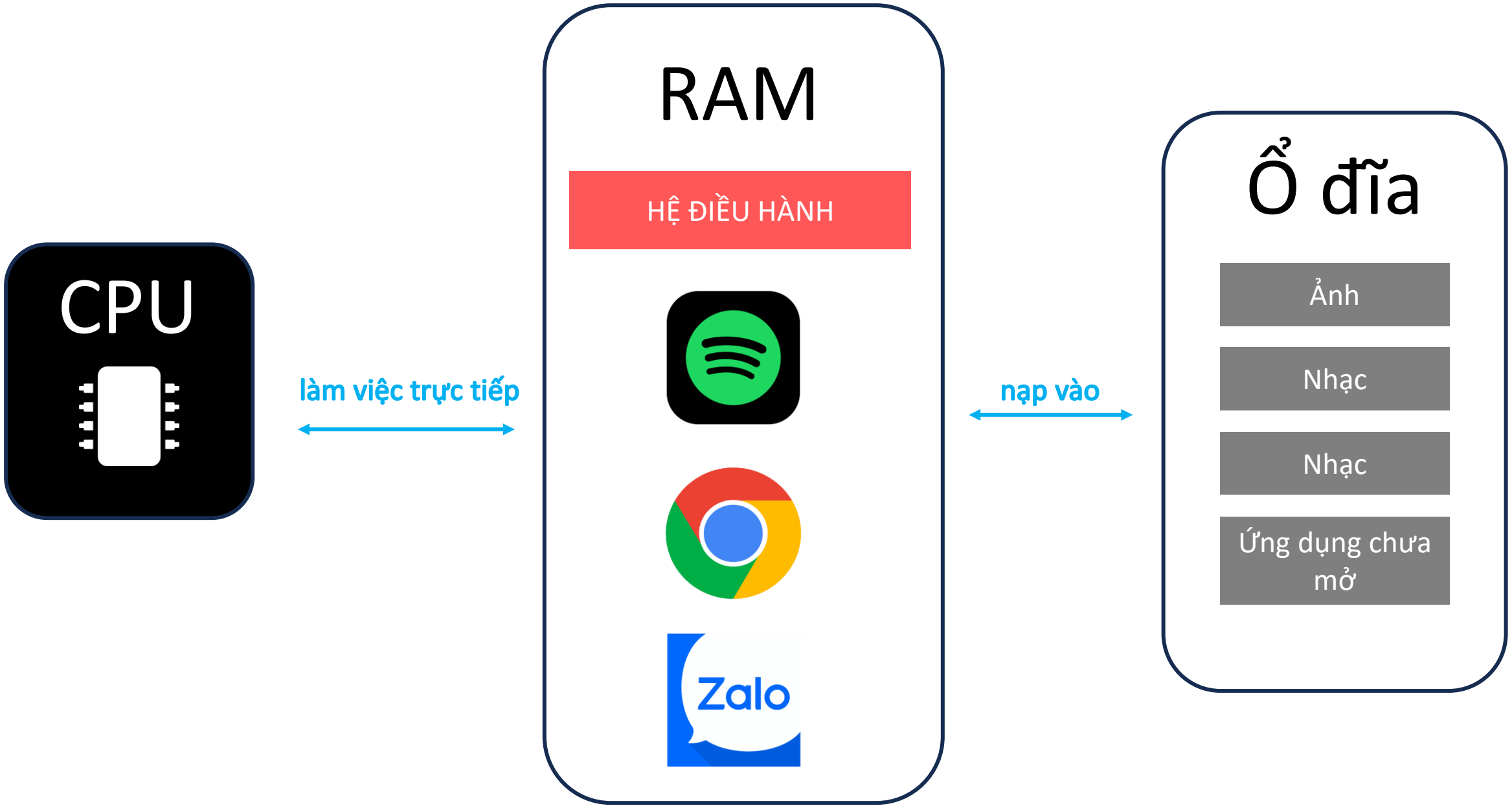
# PART 4:
# MEMORY MANAGEMENT

Nhóm 7



GV: NGUYỄN PHƯƠNG NAM


CPU
làm việc trực tiếp
RAM
HỆ ĐIỀU HÀNH
Zalo
nạp vào
Ổ đĩa
Ảnh
Nhạc
Nhạc
Ứng dụng chưa mở

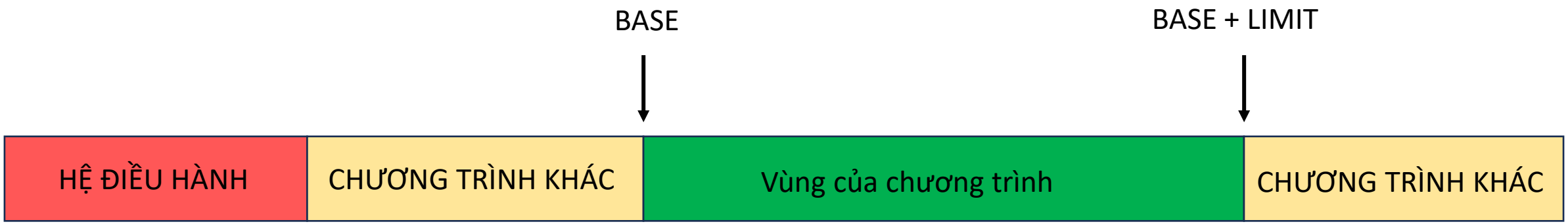
HIỆU QUẢ

AN TOÀN

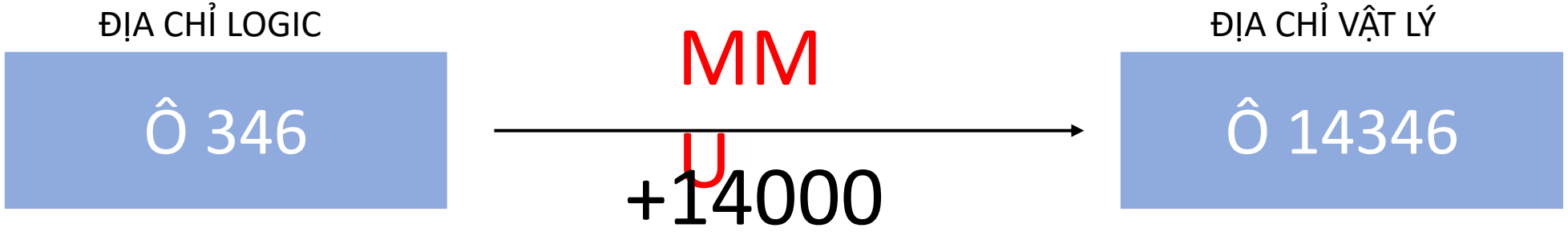
KHÔNG GIỚI HẠN

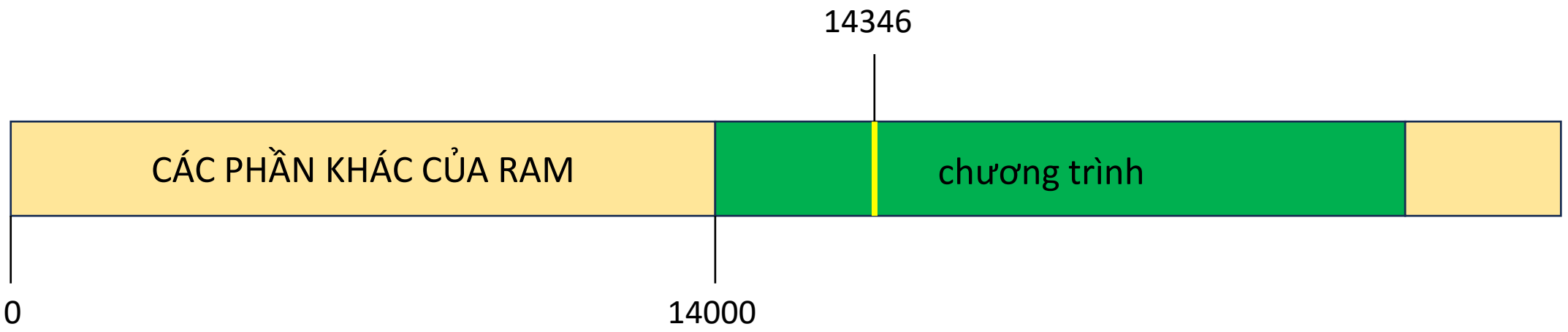


# AN TOÀN




$$BASE \leq X < BASE + LIMIT$$


ĐỊA CHỈ LOGIC
Ô 346
MM
U
+14000
ĐỊA CHỈ VẬT LÝ
Ô 14346


14346
CÁC PHẦN KHÁC CỦA RAM
chương trình
0
14000

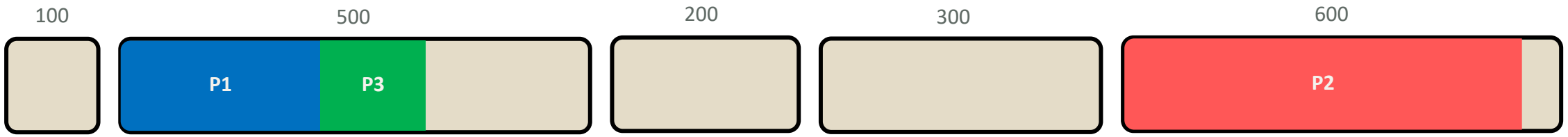
# Hiệu quả

P1 · 212 KB

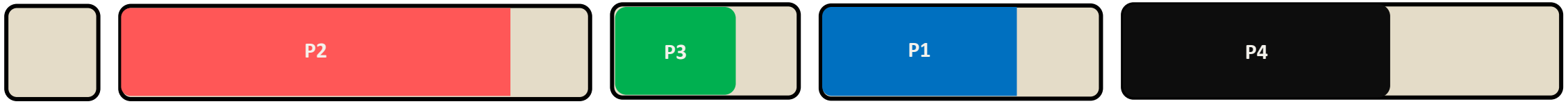
P2 · 417 KB

P3 · 112 KB

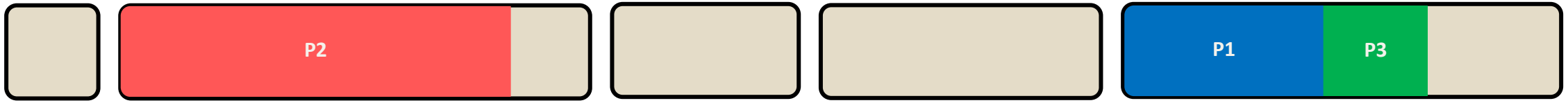
P4 · 426 KB




## First-fit




## Best-fit




## worst-fit

# Còn đủ chỗ trống, nhưng không chỗ nào đủ lớn




P4

426 KB

TỔNG CHỖ TRỐNG

959 KB

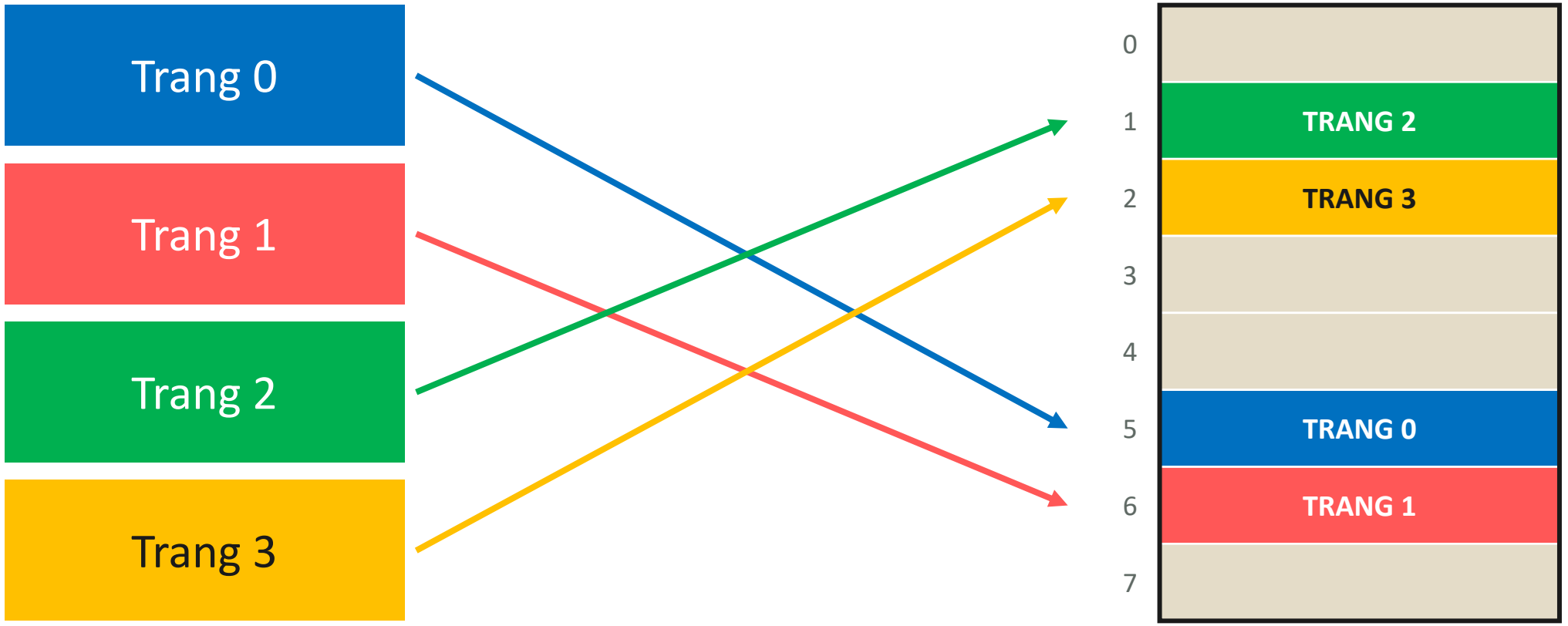
LỖ TRỐNG LỚN NHẤT

300 KB

PHÂN MẢNH NGOÀI external fragmentation

TRANG (PAGE)

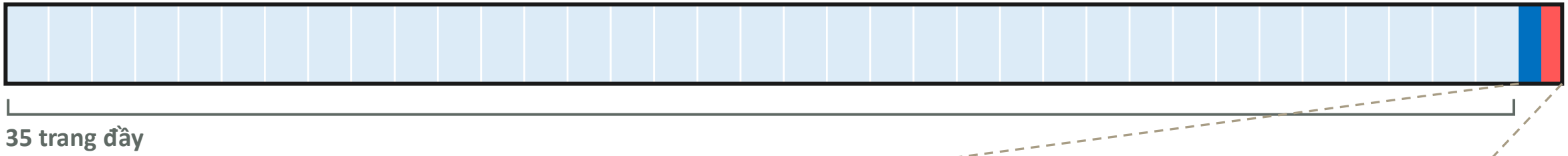
CHƯƠNG TRÌNH



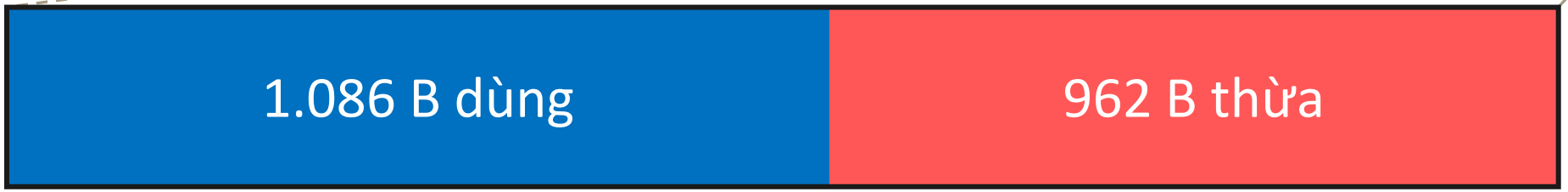

PAGE = FRAME

# Trang cuối thường thừa chỗ: phân mảnh trong

Tiến trình 72.766 byte · mỗi trang 2.048 byte



35 trang đầy




$$2.048 - 1.086 = 962 \text{ B}$$

**PHÂN MẢNH TRONG** internal fragmentation

BẢNG TRANG

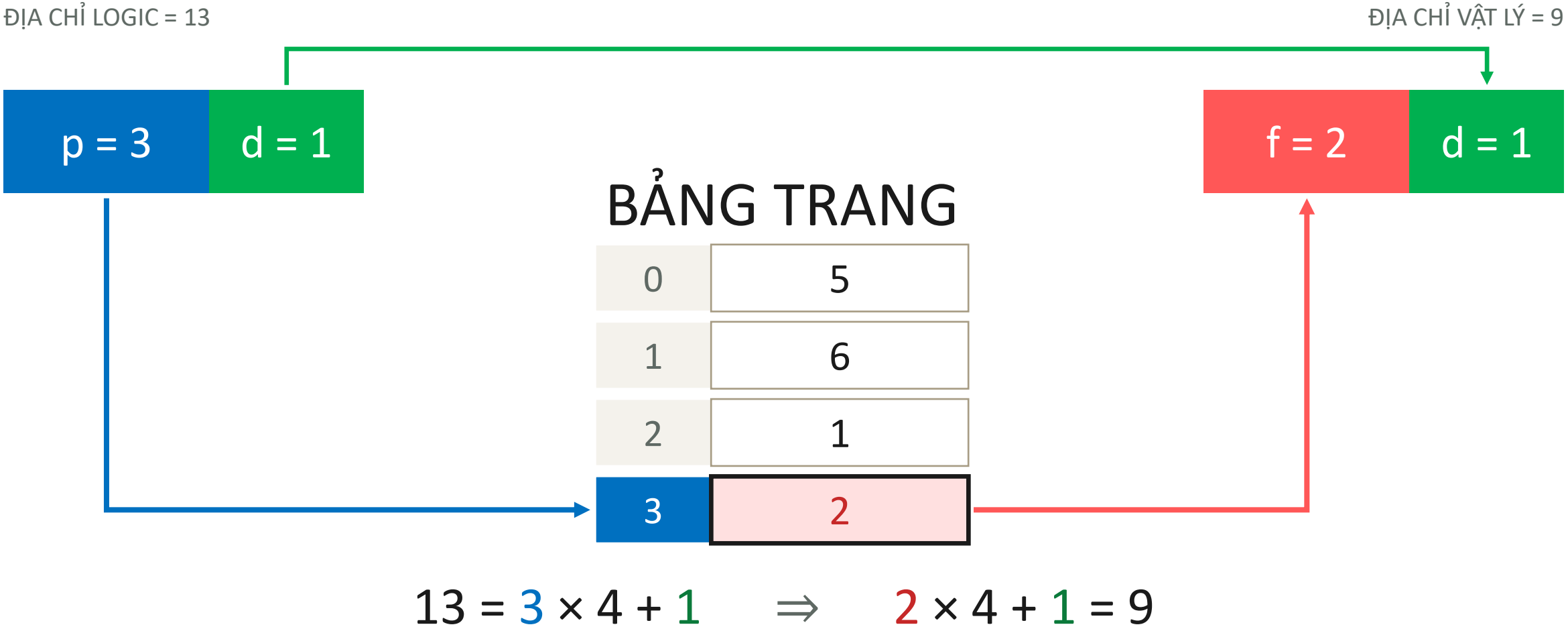


TLB



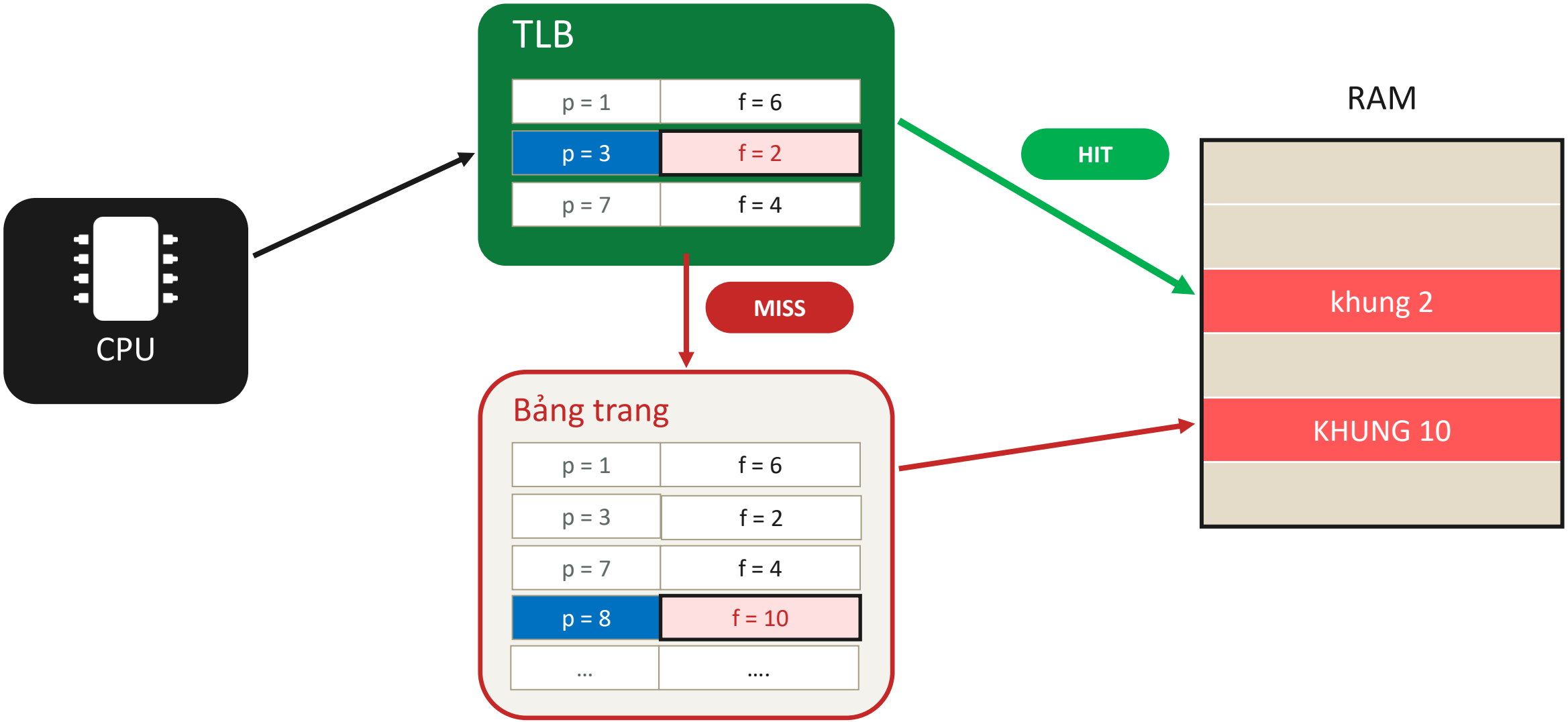
Kích thước trang: 4 byte

Đánh số từ 0

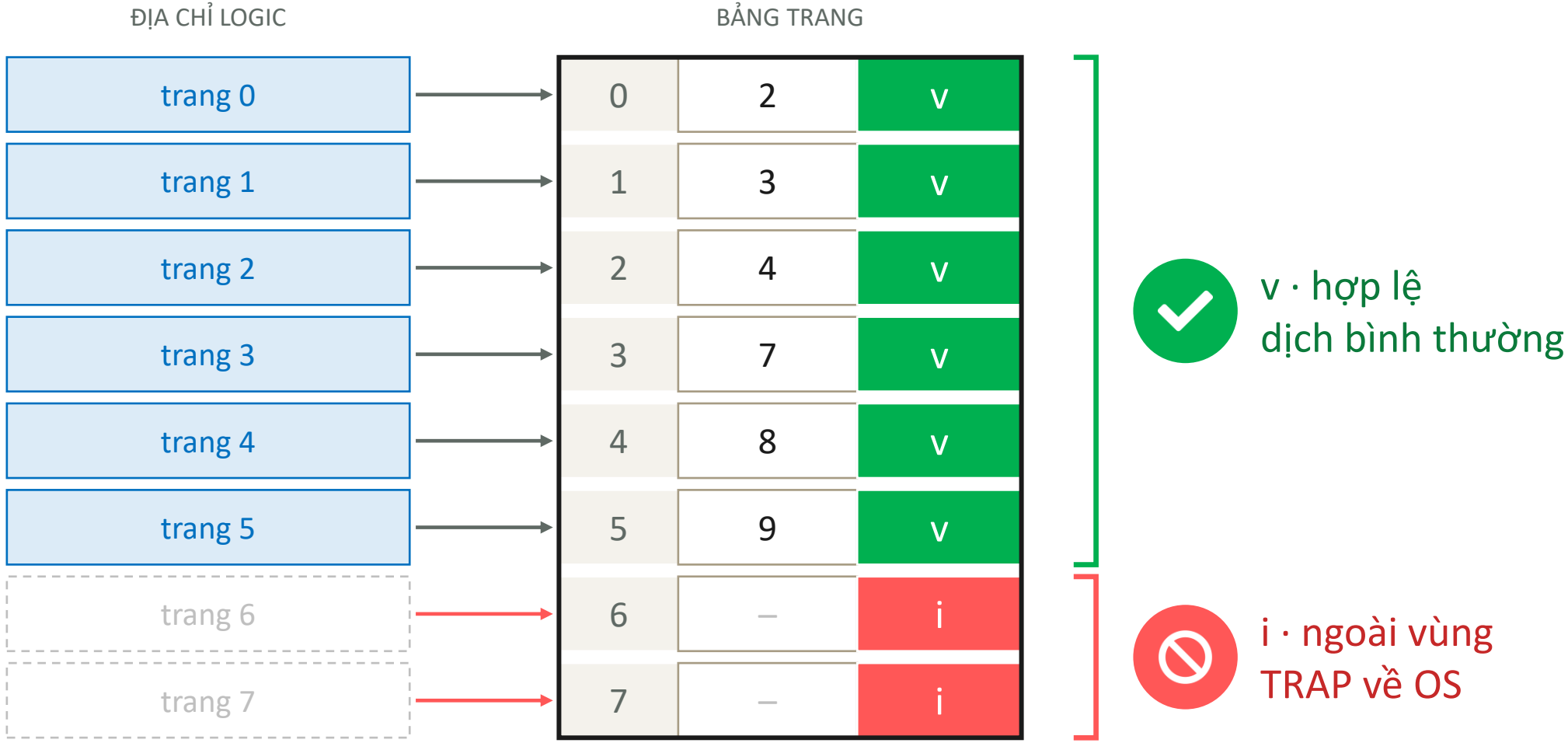


TLB: nằm trong cpu

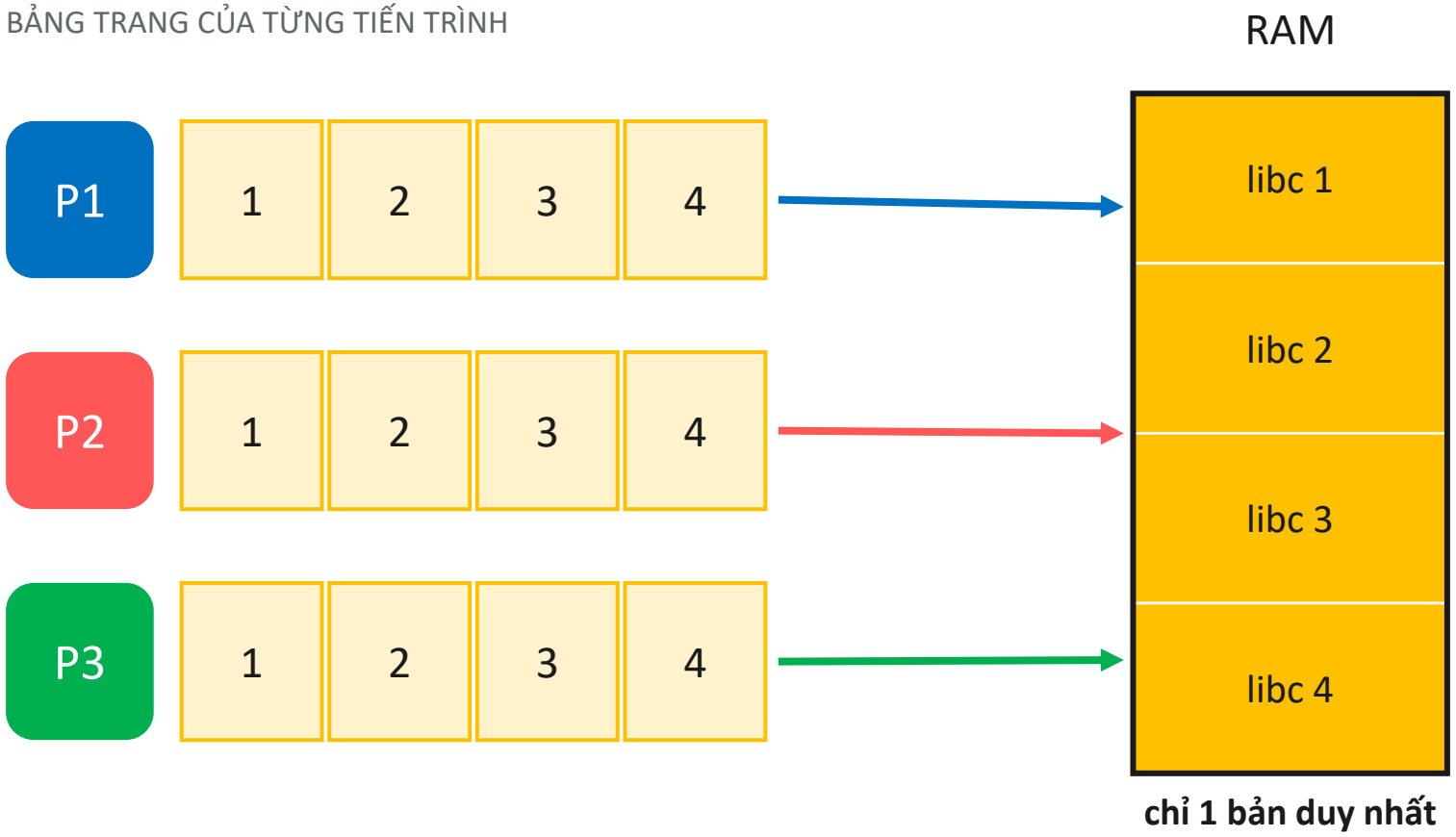
Bảng trang: nằm trong ram



# Bảng trang chặn truy cập ngoài vùng của mình

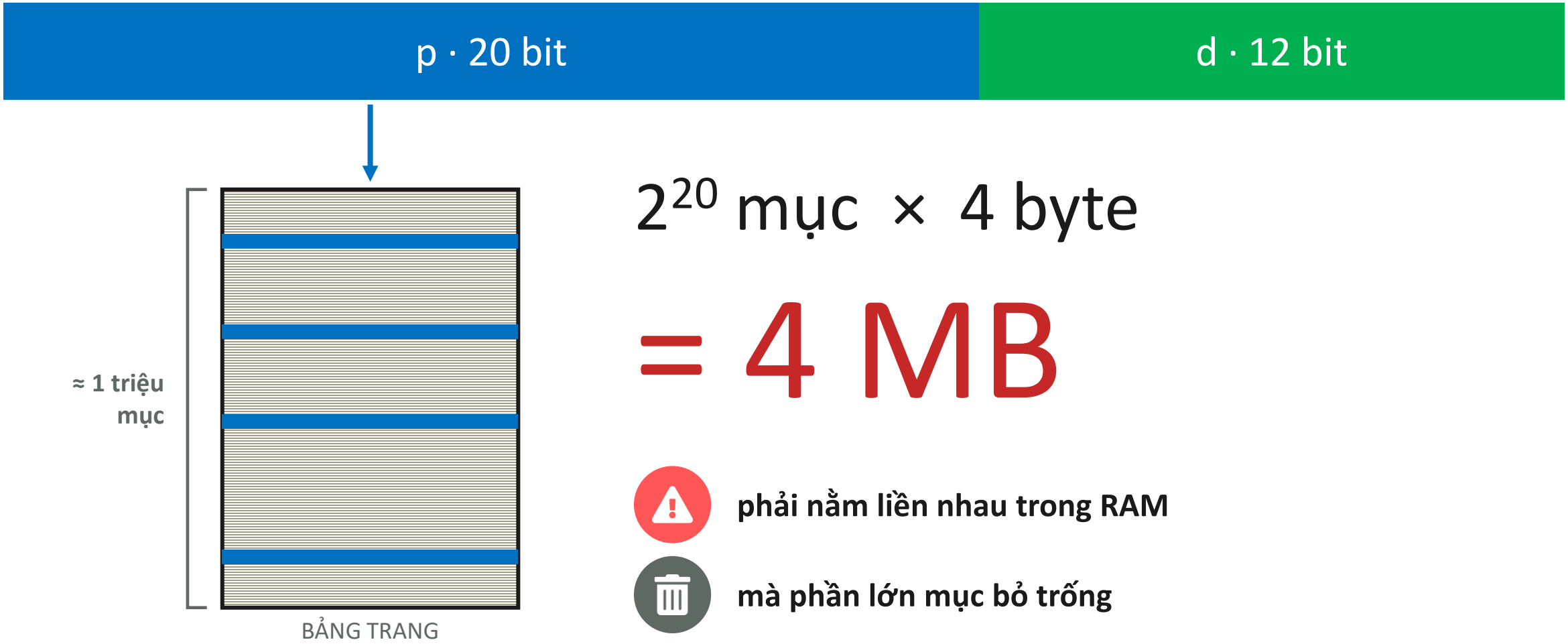


# Dùng chung trang



# Bảng trang 1 cấp: 4 MB cho mỗi tiến trình

Địa chỉ logic 32 bit · trang 4 KB ($2^{12}$ byte)



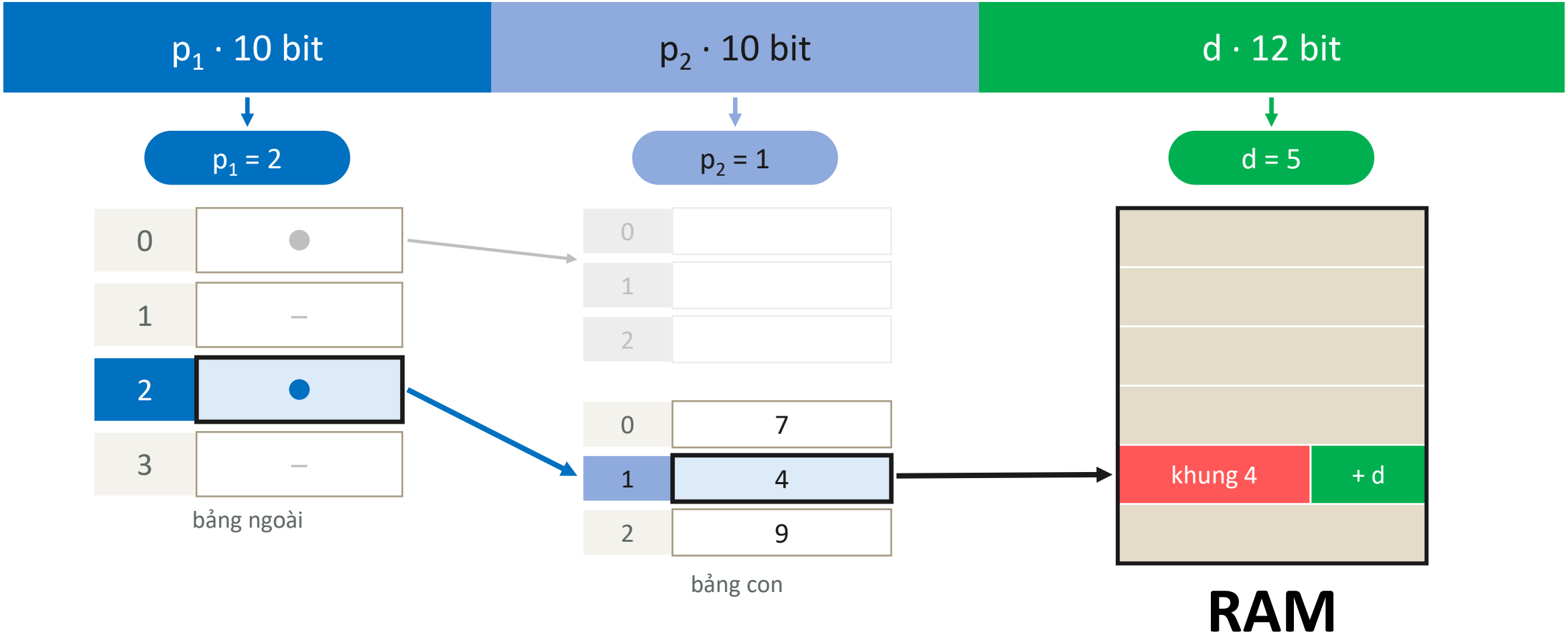

$2^{20}$ mục × 4 byte

= 4 MB

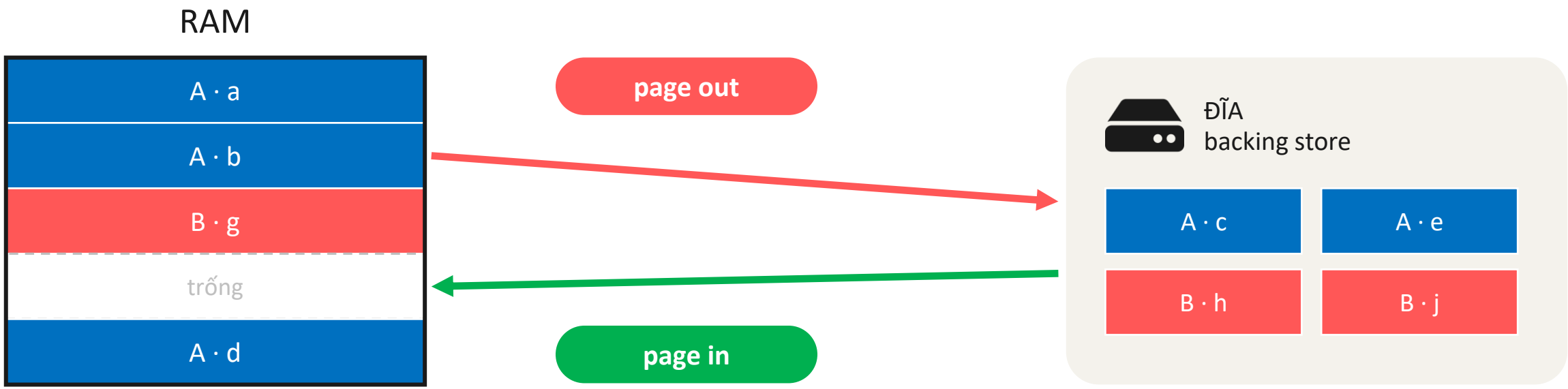
**phải nằm liền nhau trong RAM**

**mà phần lớn mục bỏ trống**

# Bảng trang hai cấp: chỉ giữ phần đang dùng



# Swapping: tạm đẩy trang ít dùng ra đĩa




**UNIX cũ**

Cả tiến trình

Từng trang

Linux, Windows

Di động

không swap: xin app nhả RAM

# Không giới hạn

# Bộ nhớ ảo

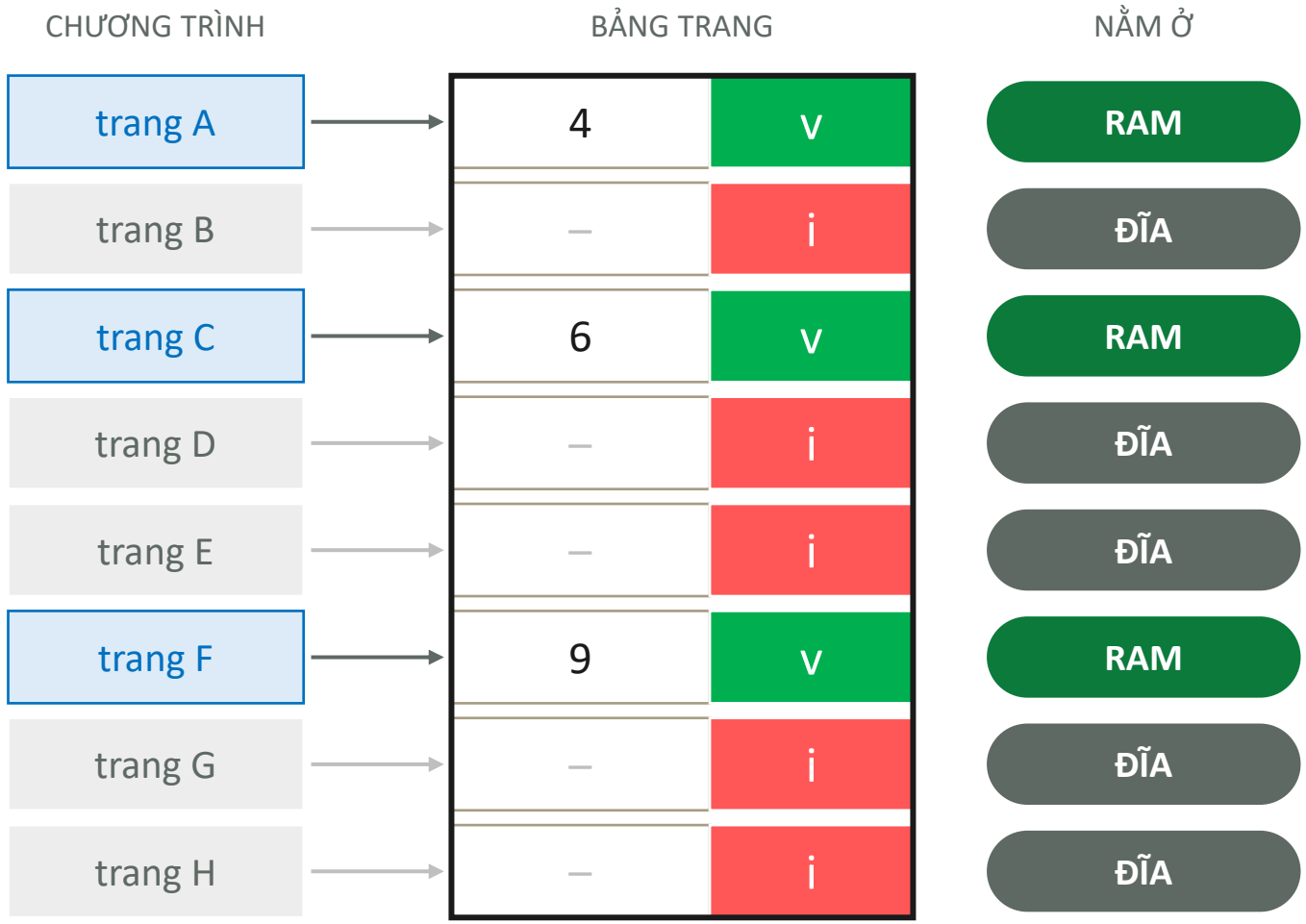
LỚN HƠN

NHIỀU HƠN

NHANH HƠN

DỄ HƠN

# Demand paging: chỉ nạp trang khi cần

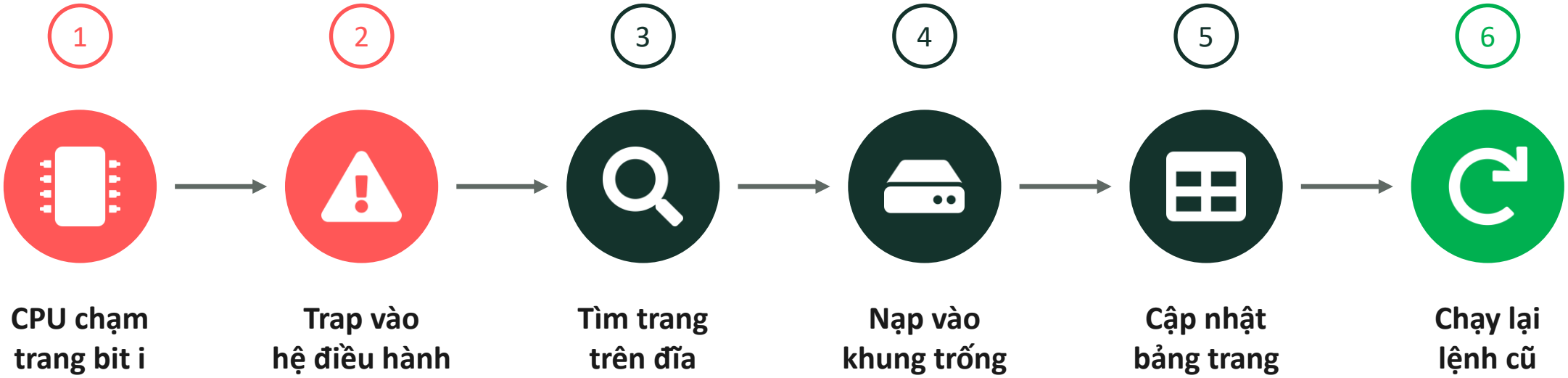



**v · trang đã nằm trong RAM**

**i · trang còn nằm trên đĩa**

**chạm vào trang i → PAGE FAULT**

# Page fault: nạp trang rồi chạy lại lệnh



# Mỗi page fault chậm gấp 40.000 lần RAM

**ĐỌC RAM**

200 ns

× 40.000

chậm hơn

**MỘT PAGE FAULT**

8 ms

$$EAT = (1 - p) \times 200 + p \times 8.000.000 \text{ ns}$$

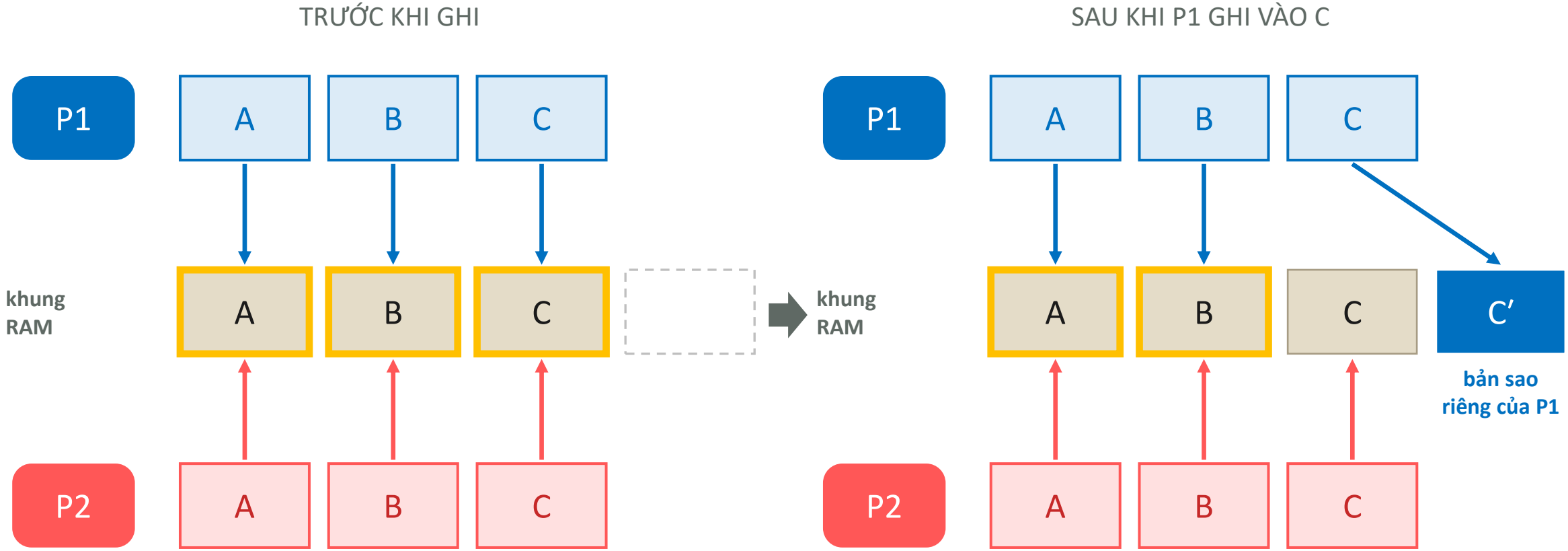
p = 1/1.000

→ **chậm gấp 40 lần**

chậm < 10%

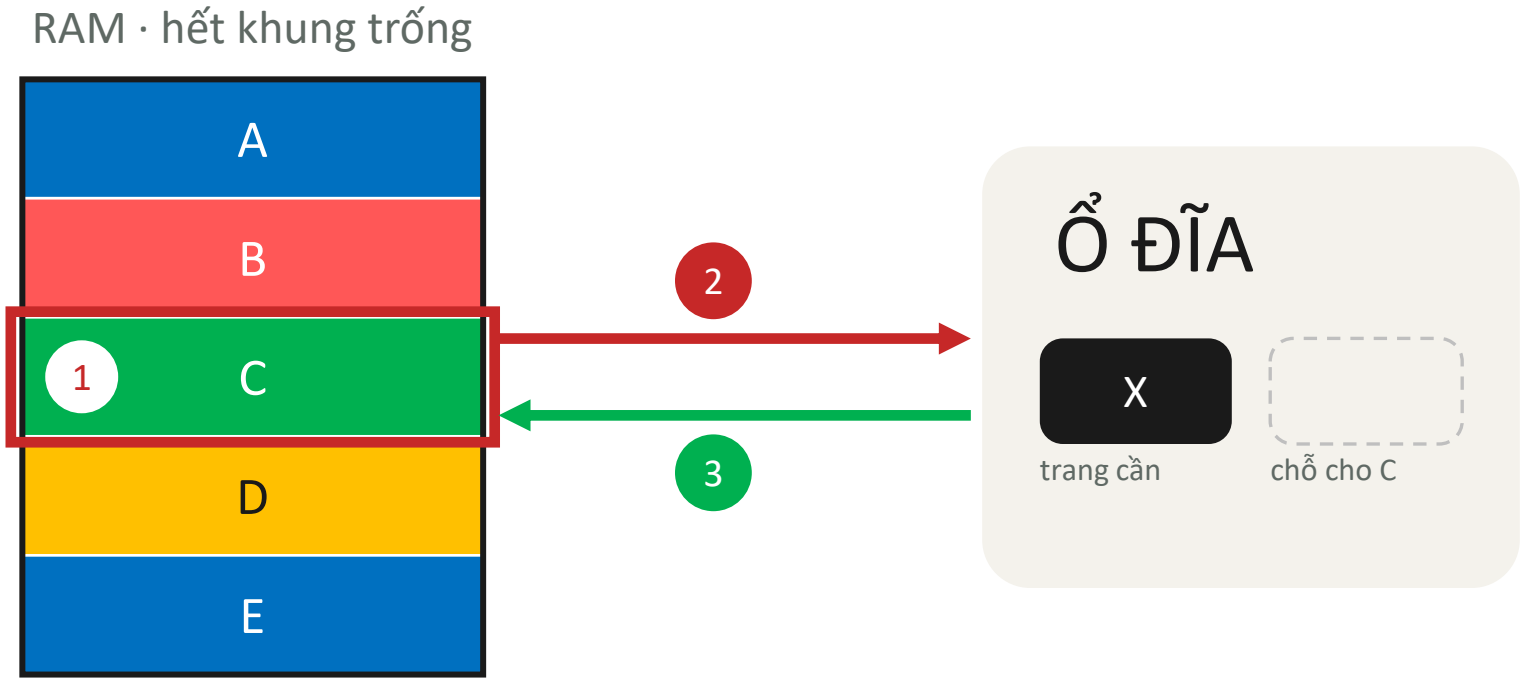
→ **p < 1/400.000**

# Copy-on-write: chỉ chép khi có người ghi




Gọi fork() rất nhanh: trang chỉ được chép khi thật sự bị ghi.

# RAM đầy? Đẩy một trang “nạn nhân” ra đĩa



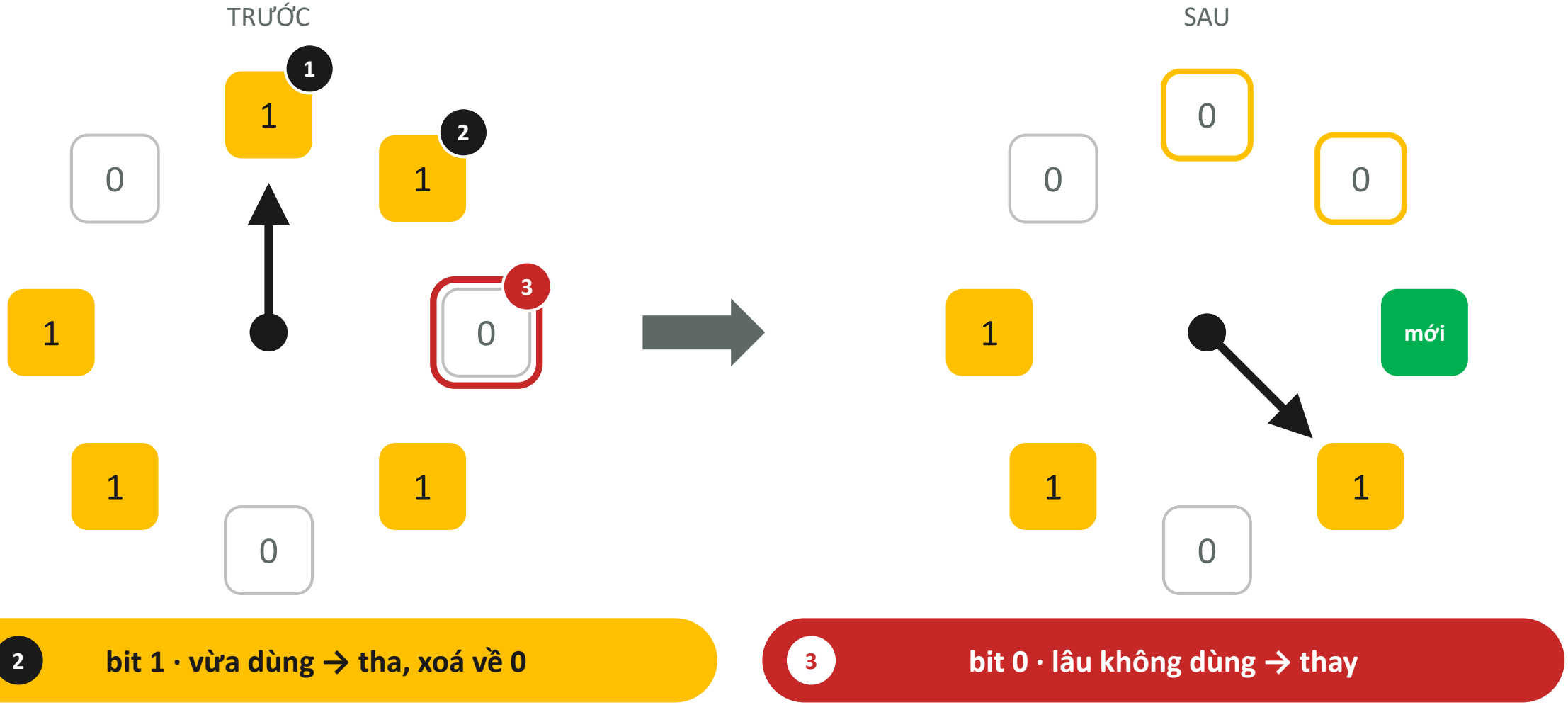

1. **Chọn nạn nhân**
2. **Ghi ra đĩa (nếu trang đã bị sửa)**
3. **Nạp trang cần dùng**
4. **Cập nhật bảng trang, chạy lại lệnh**

FIFO: đuổi trang vào RAM sớm nhất

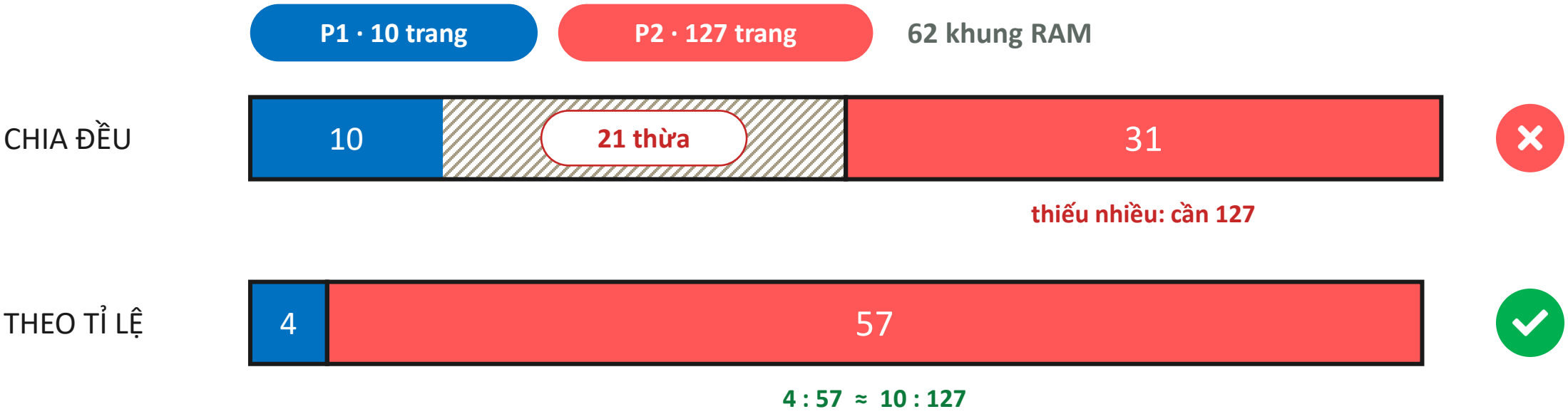
LRU: đuổi trang lâu chưa dùng nhất

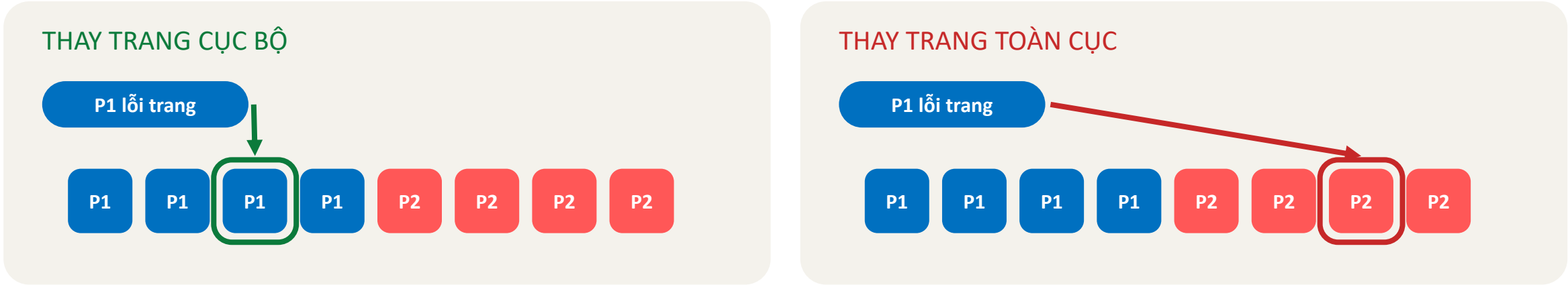
OPT: đuổi trang lâu nhất mới dùng lại

# Clock: LRU giá rẻ, chỉ cần một bit

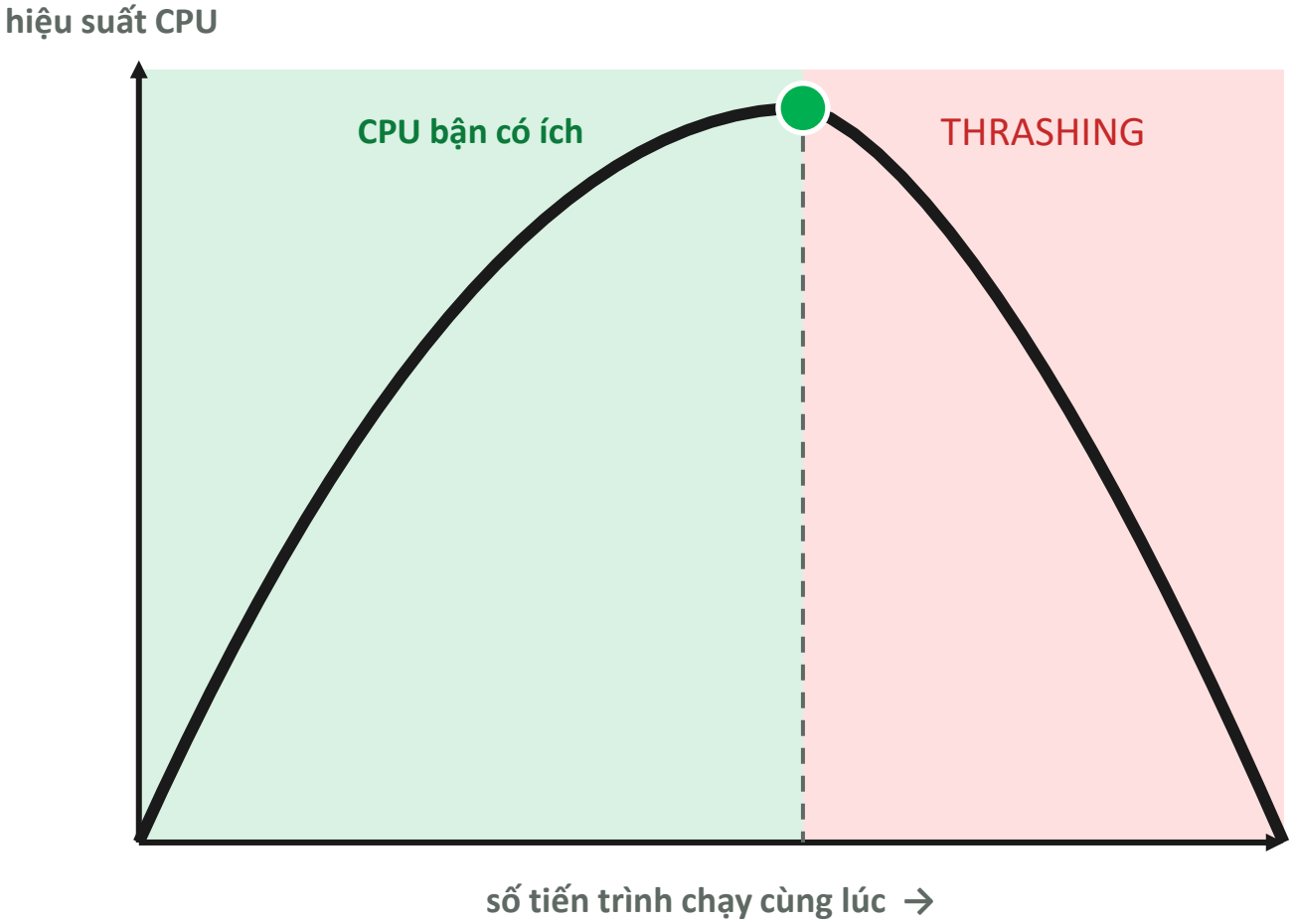


# Chia khung theo kích thước, đừng chia đều

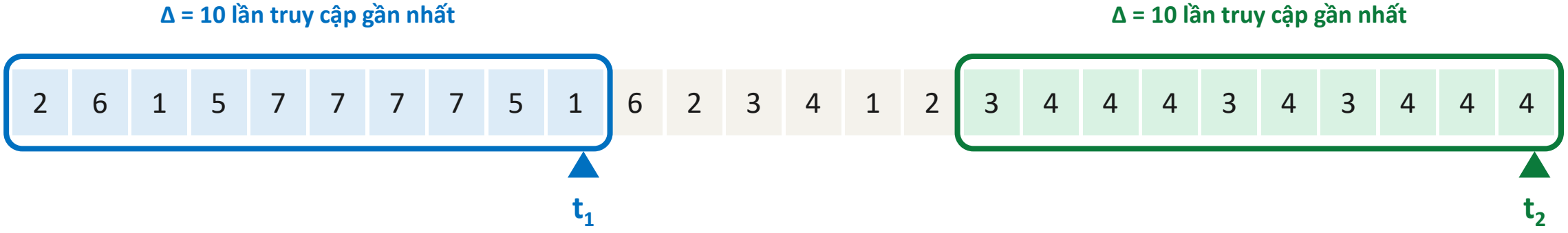


# Thrashing: bận đổi trang hơn là chạy lệnh



# Working set: giữ đủ trang đang dùng




**WS($t_1$)**

1 2 5 6 7

WSS = 5

**WS($t_2$)**

3 4

WSS = 2

**D = Σ $WSS_i$** (nhu cầu)

**m** (khung có sẵn)

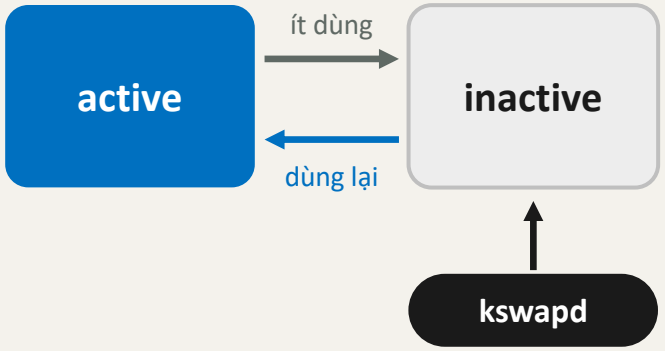


**D > m ⇒ thrashing**

HĐH tạm dừng bớt một tiến trình

# Hệ điều hành thật dùng đúng những ý này

## LINUX



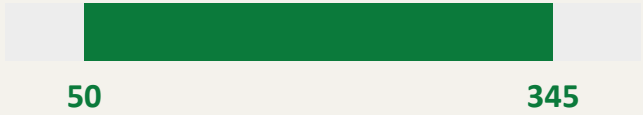

- 2 danh sách: active và inactive
- kswapd thu hồi khi RAM sắp cạn

## WINDOWS 10




nạp cả cụm quanh trang lỗi

working set mỗi tiến trình (trang)




- nạp cả cụm 3–7 trang quanh trang lỗi
- RAM cạn → cắt bớt working set

## DI ĐỘNG




3 khung → 1 khung

kích thước sau khi nén




- không swap ra flash: nén trang ngay trong RAM
- macOS và Windows 10 cũng nén

# Ba trụ cột của quản lý bộ nhớ



## AN TOÀN

- base + limit
- MMU dịch địa chỉ
- bit valid / invalid



## HIỆU QUẢ

- paging: hết lỗ trống
- TLB: tra cứu nhanh
- bảng trang nhiều cấp



## KHÔNG GIỚI HẠN

- bộ nhớ ảo, demand paging
- thay trang: LRU ≈ OPT
- tránh thrashing

RAM